Agentic Revenue, Pricing & Promotion Intelligence

The Project Bible

A complete technical account of one system: what it does, why it is shaped this way, every number it has been measured against, and the questions an interviewer will ask about it.

Domain	**CPG / Retail revenue, pricing and trade promotion**
Core commitment	Claude reasons. Deterministic models compute. Never blurred.
Scale	15 build steps, Stage 1 complete
Models	6 analytical models behind 6 agent tools
Agents	Supervisor · Critic · Recommendation, on LangGraph
Tests	1,013 passing, 2 skipped
Gates	ruff · mypy (183 files, strict) · bandit — all clean
Dataset	23.6M rows, causally simulated, hidden ground truth
Validation	Uplift recovers truth to 0.7pp across 4,417 events

The one sentence: an agentic platform where a language model decides what to investigate and when the evidence is sufficient, and where every number in the answer is produced by a deterministic model and checked against its source before a person reads it.

Contents

The PDF ships with page numbers already populated. In the .docx, a freshly generated file has an empty field — right-click the table and choose "Update Field" → "Update entire table".

00 — The System in One Picture

Mental Model

Seven analytical models with a chatbot bolted on is not an agentic system. The thing that is actually agentic here is the investigation workflow itself.

Given a business question, the system decides what to analyse, in what order, whether the evidence gathered so far is sufficient, and whether to go back and gather more. Two questions produce two completely different workflows, and a third produces no answer at all:

```
"What is next month's forecast for Product A?"
    -> forecast tool -> critic -> answer.            One tool. Stop.

"Revenue fell 12%. Cut prices or promote harder?"
    -> elasticity -> promo uplift -> optimisation -> scenarios
    -> critic -> (re-plan if evidence is thin) -> recommendation

"Why did sales of Product B collapse in July?"
    -> no registered tool separates a supply constraint from a
       demand fall -> decline, and say why.
```

Fanning out to every model for the first question would look impressive in a demo and be wrong. Selecting the **minimum sufficient workflow** is the actual skill — and the third case is the one that decides whether the first two can be trusted.

The division of labour

Claude does	Deterministic models do
Understand intent, set the objective	Forecast demand
Plan and re-plan the investigation	Estimate elasticity
Select tools	Measure promotional uplift
Interpret structured results	Optimise budget and price
Detect contradictions, judge sufficiency	Simulate scenarios
Synthesise the recommendation	Every single number

Claude never computes a business figure. This is not enforced by asking it nicely in a prompt — prompts are requests, not guarantees. It is enforced structurally: an agent's only route to a number is `AnalyticalTool.run()`, which is `@final` and always returns a `ToolResult` carrying the model name, model version and dataset version that produced the figure. A tool cannot return a bare float, and a number with no provenance cannot enter a recommendation.

The stack, bottom to top

```
INTERFACE     FastAPI · Streamlit · CLI
              one InvestigationService behind all three
GUARDRAILS    budget · output validation
              HITL interrupt · approval threshold
AGENT LAYER   Supervisor · Critic · Recommendation
              LangGraph: plan -> act -> observe -> critique
              +- bounded re-plan cycle
TOOL CONTRACT AnalyticalTool -> ToolResult
```

```
              status · provenance · confidence
              · assumptions · warnings · trace_id
SERVICES      Baseline · Forecasting · Uplift
              · Elasticity · Optimization
MODELS        baseline · forecasting · uplift · elasticity
              · optimisation · scenario
FEATURES      FeatureEngineer · availability classes
              · PointInTimeView
DATA ACCESS   DataRepository (ABC)
              DuckDB local | Databricks prod
STORAGE       Parquet gold tables (analytics)
              SQLite (investigations, traces, feedback)
```

Reasoning flows down, numbers flow up, and nothing in the top three layers may compute a figure the bottom five did not produce. That is the whole architecture in one sentence.

Interview Questions

[BASIC] Walk me through this project.

A CPG business runs a large trade promotion budget and cannot say which promotions worked. The number the industry reports — sales during the promotion minus sales before it — is wrong by roughly a factor of two, because promotions are scheduled into rising demand and because shoppers buy ahead and then buy less afterwards. This platform answers that question causally, and wraps the answer in an agent that decides which analysis the question actually needs.

[BASIC] Why not just let the LLM do the maths?

Because the failure mode is not a wrong number, it is a **confident** wrong number. An LLM that miscalculates still writes fluent justification around the result, so the error gets harder to detect rather than easier. Deterministic models are reproducible, testable against known truth, and carry provenance. The LLM's job is deciding what to compute, which is a judgement problem, not an arithmetic one.

[INTERMEDIATE] Why build the tool layer before the agents?

An agent calling an unreliable tool does not produce an unreliable answer — it produces a confident unreliable answer. I also wanted the tool contract stable before anything depended on it: agents plug into a fixed envelope rather than into the models directly, so changing an estimator never touches the agent layer.

[SENIOR] Where would this break at real scale?

Three places, in order. Investigations run **synchronously** in one process — at real concurrency that needs a job queue and a polling contract, which the API is shaped for but does not have. The app-state store is **SQLite**, which is a single-writer database. And the tools read Parquet through DuckDB on one machine; the Databricks repository exists as a designed interface with a raising body, which proves the abstraction holds but is not a migration.

Common Wrong Answers

- "The agent calculates the uplift." It cannot. The only route to a number is a tool, and the tool returns an envelope, not a float.

- “We prompt it not to hallucinate numbers.” That is one of four layers and the weakest. The load-bearing one is a check.
- “More agents is more agentic.” Three agents, and one of the four originally designed was removed because it had no work to do.

01 — Synthetic Data with Hidden Ground Truth

Mental Model

This is the cleverest part of the project and the reason everything downstream can be scored rather than admired.

The generator does not sample sales from a distribution. It **builds demand log-additively from causes** — base rate, seasonality, price elasticity, promotion lift, competitor pressure, availability — and records the true parameter of every one of those causes in a directory the application physically cannot read.

```
log(units) = base
           + elasticity * log(price / reference_price)
           + promo_lift * promo_flag
           + cross_effects + seasonality + trend
           + festival + weather + noise
```

Two consequences. Every model has a **correct answer to be measured against**, which is normally impossible — you cannot validate a causal estimate on real data because the counterfactual is absent from every dataset that will ever exist. And the log-log elasticity model is the *correctly specified* model by construction, so recovering the elasticity is a test of the estimator rather than a modelling guess.

The separation that makes it honest

`ground_truth/` is written by the generator and read only by validation code. `GROUND_TRUTH_COLUMNS` is stripped by an `observable()` function before any model sees a frame. The rule is not a convention — a model that trained on the true elasticity would score perfectly and mean nothing, so the boundary is enforced in code.

Recorded truth	Used to score
own_elasticity per product	Step 8 elasticity recovery
cross_elasticity per pair	Substitute/complement matrix
expected_log_lift per promotion	Step 7 uplift recovery
scenario_config.json (A–J)	Step 13 agent golden set
latent (unconstrained) demand	The irreducible noise floor

Deliberate realism

The generator injects the problems that make the analysis hard, because a clean dataset would validate nothing:

- **Promotions are targeted, not random.** They are scheduled into periods of high expected demand — which is exactly the confounding that makes naive uplift wrong.
- **Stockouts censor demand.** Observed sales fall while latent demand is unchanged, so a model that treats the two as the same is wrong in a costly direction.
- **Price responds to demand.** Which makes naive OLS elasticity attenuated toward zero, and creates the endogeneity Step 8 has to handle.

- **Shoppers buy ahead and then buy less.** The post-promotion dip is real, and an uplift window that stops at the promotion end date counts the pull-forward as incremental.

Interview Questions

[BASIC] Why synthetic data at all? Isn't real data better?

For a *predictive* model, yes. For a *causal* one, no — and that is the centrepiece here. You cannot validate a causal estimate on real data because the counterfactual does not exist in it. The only way to know an estimator recovers the truth is to build a world where you wrote the truth down.

[INTERMEDIATE] How do you know the generator itself is right?

A validation suite that checks business invariants (`opening + received − sold = closing`), distributional sanity, and **relationship recovery** — fitting a correctly specified model to the generated data and confirming it returns the parameters that were written in. If the generator and the recovery disagree, one of them is broken and both are mine.

[SENIOR] What does this approach fail to prove?

Volunteer this. It is the honest limit of the whole method.

Ignorability holds here because the generator's targeting rule is observable. Promotions are assigned on features the model can see, so conditional ignorability is satisfied by construction. On real data it would not be — a category manager's judgement is unobserved confounding, and no amount of estimator sophistication fixes that. So this proves "the estimator is correct given the assumptions" and says nothing about "the assumptions hold". Those are different claims and I keep them apart in the docs.

Common Wrong Answers

- "Synthetic data means the results are fake." The results are about the estimator, not the business. That is what they are for.
- "Ground truth is in the training data." It is structurally excluded, and the test that proves it is a real test, not a comment.

02 — Data Access, Contracts and the Feature Layer

Mental Model

Leakage prevention here is structural, not procedural. You cannot write the leaking query, because the object you would write it against does not expose tomorrow.

`PointInTimeView` takes an as-of date and filters every table to what was **knowable on that date**, respecting each table's own availability lag. That last part matters and is the bit most implementations get wrong.

Availability is per-table, not global

A single global cutoff is wrong because tables land at different times. Point-of-sale is available next day. Inventory reconciles weekly. Competitor pricing arrives on a scrape schedule. A feature built at as-of date D using inventory that will not exist until D+7 is leakage that no date filter on the sales table would catch.

Table	Availability class	Effect at as-of D
sales	next-day	visible to D−1
inventory	weekly reconcile	visible to last week boundary
competitor_price	scrape lag	visible to D−lag
promotion_plan	known forward	visible into the future, legitimately

Promotion plans are the interesting exception: they are **known in advance**, so a forecast may legitimately use next month's planned promotions. Treating all future information as leakage would have removed the single most predictive input a demand forecast has.

Interview Questions

[BASIC] What is data leakage in a forecasting context?

Using information at training time that would not have been available at prediction time. The model scores well offline and collapses in production, and the gap is invisible until it is expensive.

[INTERMEDIATE] Give me a leakage bug that a date filter would not catch.

A rolling 7-day mean computed over a window that **includes the target day**. Every row is dated correctly and the feature is still poisoned. The fix is that lag and rolling features are constructed by a layer that shifts before it aggregates, not by whoever is writing the model.

[SENIOR] How do you prove leakage prevention actually works?

A test that builds the same feature frame at two as-of dates and asserts the earlier one is a strict prefix of the later one — no row changes retrospectively. A test that a table added after the as-of date is invisible. And a test that the promotion-plan exception is *deliberate*, so nobody later “fixes” it into a regression.

03 — Baseline Sales and Demand Forecasting

Mental Model

Two models that sound similar and answer different questions. The baseline is a nowcast of a counterfactual; the forecast is a prediction of the future.

	Baseline	Forecast
Question	What would normal sales be?	What will sales be?
Time	Same day, no promotion	Days ahead
Consumed by	Uplift, root cause, scenarios	Planning, the agent
Framing	Counterfactual	Predictive

Baseline: two approaches, measured

Approach C trains only on non-promotional rows, so the model has literally never seen a promotion and its prediction *is* the no-promotion expectation. **Approach B** trains on everything with promotion features, then predicts with them zeroed.

Result	Value
Selected model	LightGBM, Approach C (exclude)
WMAPE	40.4%
Irreducible noise floor	35.0% (from latent demand)
Ratio to floor	1.15×
Margin over Approach B	0.2 percentage points
Bias	over-predicts by +6.7%

Two honest readings that belong in the answer, not a footnote.

A model scoring 40.4% WMAPE is **1.15× the noise floor, not 60% accurate**. Without the floor you cannot tell an excellent model from a mediocre one and you will burn months chasing error that is not there. And a 0.2-point margin is **far too narrow to call a general result** — Approach C won on this dataset, and I would not claim more than that.

The bias matters downstream: a baseline that over-predicts by +6.7% makes uplift measured against it **understated** by roughly that much. The document flags the tension and hands the decision to whoever owns the uplift numbers rather than quietly correcting it.

Forecasting: direct multi-step

One global model, horizon `h` as a feature, training row = `(origin t, horizon h, target = units at t+h)`.

Alternative	Why not
Recursive one-step-ahead	Feature-distribution collapse — feeding conditional means forward drives rolling std to zero, so by h=30 the model sees inputs it never saw in training
Per-horizon models	Cannot produce a daily path; nested totals are mutually incoherent
One model per series	Thousands of models, no cross-series learning, cold start on any new product

Intervals are **conformal**, which is distribution-free with a finite-sample coverage guarantee. The property that matters is not the guarantee though — it is that achieved coverage is *measured on test data and reported whatever it turns out to be*, rather than asserted.

Interview Questions

[INTERMEDIATE] Why not use the forecast as the promotion baseline?

Because the forecast is trained on data that includes promotions, so it partly predicts the promotion's own effect. Subtracting it from actuals would remove part of the thing being measured. The baseline has to be a model that has never seen a promotion.

[SENIOR] Your forecast intervals are too narrow in production. Diagnose.

First check whether the calibration set is representative — conformal coverage is only valid under exchangeability, and a promotional period calibrated on a quiet one breaks that. Then check for clustering: if residuals correlate within a product or a store, the effective sample size is the number of *series*, not rows, and the interval is computed on a sample size you do not have. That exact bug appears in the uplift chapter with numbers.

04 — Promotional Uplift: Causal Inference

The centrepiece. If there is time for one chapter, it is this one.

Mental Model

This is not a prediction problem. You are estimating what would have happened without the promotion — a quantity that is absent from every dataset that will ever exist.

So you cannot validate it by holding out data. Held-out data tells you how well you predict what *did* happen; it says nothing about what *would* have happened. That single fact drives every design decision in this chapter, including the synthetic dataset two chapters back.

The worked example

Have this ready. It lands, because the arithmetic is trivial and the point is not.

```
Sales during promotion       1,000 units
Sales the week before          700 units
"Uplift"                       300 units   <- the industry number

But:
  demand was already rising      +120 units   (seasonality)
  promotions are scheduled
    into strong weeks            +90  units   (targeting/confounding)
  shoppers bought ahead          -60  units   (pull-forward, shows up later)

True incremental                ~150 units
The reported number is 2x the real one.
```

Treatment is the whole event, not the flag

A row-level `promo_flag` is the wrong estimand. The promotion has an **anticipation** period (shoppers wait), a **live** period, and a **post** dip (pantry loading). Treating only live days as treated counts the dip as an unrelated slump and the anticipation as normal trade.

So rows are classified into `TREATED`, `WASHOUT`, `CONTROL` and `EXCLUDED`, and treated rows anchor their features at the **event start** rather than the row date — otherwise a feature computed on day 12 of a promotion is contaminated by the promotion's own first 11 days.

A promoted day misfiled as a control raises the baseline, so the bias is UNDERSTATED uplift. The code originally documented this backwards.

Six estimators, and why more than one

Estimator	Identifying assumption	Role
Naive before/after	None — it is the wrong answer	Quantifies the bias avoided
Baseline difference	Baseline model is correct	Fast, model-dependent
Difference-in-differences	Parallel trends	Standard, and see below
IPW (Hajek-normalised)	Conditional ignorability + positivity	Weighting
AIPW (doubly robust)	Either outcome or propensity correct	Primary
DR-learner	Same, plus a CATE model	Heterogeneity

AIPW is doubly robust: it is consistent if *either* the outcome model *or* the propensity model is correctly specified, not both. That is the reason it is primary. Cross-fitting removes the bias from using the same data to fit the nuisance models and estimate the effect.

The validation results

Six synthetic scenarios where the effect is known exactly:

Scenario	True	Naive	AIPW	Recovered
confounded_null	0.0%	+34.9%	+0.2%	yes
clean_positive	known	—	within tolerance	yes
pull_forward	known	overstated	within tolerance	yes
heterogeneous	known	—	within tolerance	yes
low_overlap	known	—	within tolerance	yes
deep_discount	known	—	within tolerance	yes

confounded_null is the one to quote: promotions that did nothing at all, on days that would have sold well anyway. Naive reports +34.9% of pure fiction. AIPW reports +0.2%.

Then on the real generated dataset, scored against the generator's own recorded parameters across **4,417 promotion events**: expected **+71.3%**, estimated **+72.0%**. An error of **0.7 percentage points**.

Three debugging stories

Rehearse these. They are what separate a candidate who ran a library from one who debugged one.

1. −424% against a true +65%

I cross-fitted on contiguous **date** blocks. The linear `time_index` feature then had to extrapolate outside every fold's training range, the propensity model hit its clip boundaries, and control weights summed to **43× the treated count**. The estimate was not slightly wrong, it had the wrong sign and an absurd magnitude.

Fixed by clustering folds on **series** rather than dates. Then I added a permanent guard, because a bug that produced −424% once will produce −40% silently later:

```
E[(1-T) * e/(1-e)]  ==  P(T=1)     warn if ratio outside [0.7, 1.4]
```

2. Intervals three to five times too narrow

Coverage of known truth was **4/6** while every point estimate was within 2–5 percentage points. That combination is diagnostic: the estimates are fine and the *uncertainty* is wrong. I had computed standard errors as though rows were independent, when residuals cluster within a product-store listing. Clustering on the listing widened intervals 3–5× and brought coverage to **6/6**.

3. DiD passed its own test and was still 21 points out

The pre-trend test returned **p = 0.64** — no evidence against parallel trends — and the estimate was 21 points from truth. A pre-trend test has low power and tests a *necessary* condition, not a sufficient one. Passing it is not evidence the assumption holds.

Interview Questions

[BASIC] What is ATT and why is it the right estimand here?

Average Treatment effect on the Treated: the effect among the promotions that actually ran. That is the business question — "was this promotion worth it" — not "what if we promoted everything", which is ATE and which the data cannot support anyway because positivity fails for products that are never promoted.

[INTERMEDIATE] Why not just control for discount depth?

This is the best single question in the whole document.

Because discount depth is a **mediator**, not a confounder. The promotion causes the discount, and the discount causes the sales lift. Conditioning on it blocks the causal path you are trying to measure.

Measured across the same 4,417 events: the mechanic channel is **+17.7%** and the price channel is **+45.6%**. A model that conditions on discount blocks the larger path and reports **+17.7% as the whole effect** — a number that is wrong by a factor of four, looks entirely reasonable, and has a plausible story attached.

[INTERMEDIATE] What is positivity and how did you check it?

Every unit must have a non-zero probability of both treatment and control given its covariates. Checked by inspecting the propensity distribution for mass at the boundaries and by standardised mean difference on covariates after weighting. Weights are stabilised at the 99th percentile — before that, a single row at e = 0.98 carried a weight of 49 and pushed covariate balance from +0.27 to −0.38, which is overshoot, not correction.

[SENIOR] How would you deploy this against real data?

I would not present a point estimate first. I would run the placebo tests, report the overlap diagnostics, and show the naive number alongside the causal one so the size of the correction is visible. Then I would say plainly that ignorability is **assumed, not verified**, and name the most likely unobserved confounder — the category manager's judgement about which products deserve support. The estimate is only as good as that assumption, and pretending otherwise is how these numbers lose credibility the first time someone checks one.

Honest limitations

Volunteer all four before you are asked.

- **ROI on this dataset is an artefact.** Promotional spend runs ~20× the achievable margin at product-store grain, so 96.8% of events appear value-destroying. The uplift is sound; the ROI is not interpretable, and the optimiser is therefore validated on ranking and constraint satisfaction rather than absolute profit.

- **Cannibalisation is not deducted**, so profit is an upper bound on category profit.
- **Ignorability holds because the generator's targeting is observable.** That is a property of the data, not an achievement.
- **The baseline's +6.7% over-prediction** understates uplift by roughly that much.

05 — Price Elasticity: Own and Cross

Mental Model

Price moves with demand. That single fact is why the naive elasticity calculation is wrong, and in a direction you can predict.

If prices are raised when demand is strong, a regression of quantity on price sees high prices alongside high volumes and concludes demand is less price-sensitive than it is. The bias is **toward zero** — attenuation — and it makes every product look safe to raise the price on.

Four estimators, two of them not selectable

Method	Handles	Selectable
naive_ols	Nothing. The wrong answer, kept as a comparison	no
panel_fe	Product and store-time fixed effects	yes — preferred
randomised	Where price variation is experimental	yes
iv_2sls	Endogeneity, via a cost instrument	no — see below

Fixed effects absorb anything constant within a product or within a store-time cell. That fixes **seasonal** endogeneity — the whole category being priced up at Diwali — and it cannot fix **idiosyncratic** endogeneity, where one product's price responds to its own demand shock. The test suite has a fixture for each, and the second one asserts that FE *does not* fix it.

The instrument that failed a test it appeared to pass

Strong first-stage F, and still invalid. This is the story worth telling.

The commodity cost index is a textbook instrument: it shifts price and has no direct path to demand. Median first-stage **F = 484**, far above any weak-instrument threshold. And 2SLS still could not identify the elasticity, because the cost index **varies only at category×date** — it has no cross-sectional variation, so within a category-date cell there is nothing left for it to explain.

The lesson is that a strong F statistic is **necessary and not sufficient**. `instrument_diagnostics()` now checks directly for zero cross-sectional variation, and 2SLS is computed and marked not-selectable so the comparison stays visible without being trusted.

Cross-price and a bug worth knowing

Substitutes and complements come from pairwise regressions with Benjamini–Hochberg correction, because testing hundreds of pairs at $\alpha = 0.05$ produces a substitute list that is mostly noise.

The bug: dropping the wrong panel's promotions lost the true substitute.

I dropped promoted rows from **both** the focal and the source product to avoid promotional contamination. That removed 29% of the identifying variation — log-price standard deviation fell from 0.155 to 0.120 — and the real substitute stopped being detectable. The fix: the focal panel

drops promotions, the source panel keeps them, because the source product's price movement is the *signal*, not the contamination.

Interview Questions

[BASIC] What does an elasticity of −1.8 mean?

A 1% price rise reduces demand by 1.8%. Because |e| > 1 the product is **elastic**, so a price rise *reduces* revenue. Below 1 it is inelastic and a price rise raises revenue. That threshold is the entire pricing decision.

[INTERMEDIATE] Derive the profit-maximising price.

```
p* = c * e / (1 + e)      for e < -1, with c = marginal cost
```

Which is why the optimiser refuses to return a single number when the elasticity's confidence interval is wide: a point optimum computed from an uncertain slope is false precision, so it returns a **range** in which every price is roughly equivalent.

[SENIOR] Your elasticity says +0.3. What do you do?

Not report it. A positive own-price elasticity means either the specification is wrong, the price variation is entirely endogenous, or there is a Giffen/Veblen story that is almost never true for CPG. I would check overlap and identifying variation first, and if the sign survives, report that the elasticity **could not be identified** rather than publish a number that implies raising prices increases demand.

06 — Optimisation and Scenario Simulation

Mental Model

Three capabilities that share a shape: consume the causal estimates and project or optimise under constraints. None of them fits anything.

Budget allocation

Allocate a fixed trade budget across candidate promotions to maximise incremental profit, subject to per-region minimums, per-retailer caps and margin floors. Diminishing returns are modelled with a **concave piecewise-linear** approximation so the solution stays an LP that OR-Tools GLOP solves exactly, rather than pouring the entire budget into the single highest-ROI cell.

Infeasibility is a FINDING, not an error: "your minimum spends already exceed the budget" is the most useful thing the optimiser can say, and an exception would throw it away.

Two bugs worth telling

The price grid pinned every recommendation to its edge. The grid was ±15%, and the optimum for an elasticity of −2 is +20% — so every answer landed on the boundary and looked like a recommendation rather than an artefact of the search range. Widened to ±30% with an explicit warning when the optimum still lands on the edge.

A constraint matching no candidate was silently dropped and then reported as binding. The worst possible combination: it claimed to have constrained something it had never seen. The event table had no `region` column, so a regional minimum matched nothing. Now unmatched constraints produce a warning and only *applied* limits can be reported as binding.

Scenario simulation

Composes elasticity, cross-price and uplift in log space to project combined levers. Two properties that matter more than the projection:

- **Confidence is the weakest component, not an average.** A scenario composed of a solid elasticity and a shaky uplift is a shaky scenario. Averaging would hide exactly the component that should stop you acting.
- **A lever that is not modelled is reported, never silently ignored.** The API accepts an inventory change the engine cannot project; it comes back as a warning, because a projection that quietly dropped it would answer a different question than the one asked.

Interview Questions

[INTERMEDIATE] Why linear programming rather than a heuristic?

Because the constraints are real business rules and an LP gives you a **proven optimum plus a certificate of infeasibility**. A greedy heuristic gives you a plausible answer and no way to tell whether a better one exists, which is precisely the question a finance director asks.

[SENIOR] Your optimiser recommends putting the whole budget on one SKU.

That is the diminishing-returns model failing, not the optimiser. With a linear objective the LP will always corner-solve. The piecewise-linear concave segments exist precisely to stop it — and if it still corners, the segments are too coarse or the ROI estimates are too confident. On this dataset I would also check whether the ROI artefact is driving it.

07 — The Tool Contract

Mental Model

The contract is the entire safety argument. Everything above it is reasoning; everything below it is arithmetic; the envelope is what stops the two from mixing.

```
class ToolResult:
    status          success | partial | error
    result          the payload, or None
    error           structured, with a `recoverable` flag
    provenance      model_name · model_version · dataset_version
    confidence      float | None  -- None is a legitimate answer
    assumptions     list[str]  -- conditions the number depends on
    warnings        list[str]
    trace_id        correlates to every log line in the investigation
```

`AnalyticalTool.run()` is `@final` and **never raises** — every outcome is a `ToolResult`. That matters because an exception crossing into the agent loop would either crash the investigation or, worse, be caught generically and turned into a plausible-sounding gap in the evidence.

What the design forbids

- A tool cannot return a bare float. There is nowhere to put it.
- A tool cannot return a number without provenance.
- An agent never sees a DataFrame, a model object, a file path or a database connection. It sees names and structured results.
- `confidence=None` is allowed and meaningful — inventing a confidence score to fill a field is worse than admitting there is not one.

Registered tools

Tool	Answers
forecast_demand	What will demand be over 7–90 days?
estimate_promo_uplift	What did this promotion actually cause?
estimate_price_elasticity	How price-sensitive is this, and what competes with it?
allocate_promotion_budget	Where should the next unit of budget go?
optimize_price	What price, and how confident is that?
simulate_scenario	What happens if we pull these levers together?

baseline_sales is deliberately NOT registered. It answers "what would normal sales have been", which is an input to uplift rather than a question anyone asks — exposing it would invite an agent to compute uplift itself by subtraction, which is the naive estimate the whole causal layer exists to avoid.

Interview Questions

[INTERMEDIATE] Why do tool descriptions contain warnings to the model?

Because tool selection is mostly a prompt problem. The uplift tool's description ends with “do NOT compute uplift yourself from sales figures — the counterfactual is not in the data”. That sentence is in the description rather than the system prompt because it is needed **at the point of choosing**, and a model reading a tool manifest is deciding right there.

[SENIOR] How do you stop tool sprawl as the model count grows?

Registration is explicit and manual — `build_default_registry` names every tool. A tool becoming callable by an agent is a decision someone made, not a side effect of a file existing. Auto-discovery would make the agent's capability surface change silently on merge.

08 — LLM Providers: Claude and the Offline Stub

Structured output via forced tool-calling

Not JSON parsing. The target Pydantic model's JSON schema becomes a single tool's `input_schema`, and `tool_choice` forces the model to call it. Materially more reliable than asking for JSON and hoping, and the failure mode is a clean exception rather than a half-parsed dict.

```
schema = _tool_schema(response_model)      # $refs flattened inline
raw = self._call(
    messages,
    tools=[{"name": "emit_structured_response", "input_schema": schema}],
    tool_choice={"type": "tool", "name": "emit_structured_response"},
)
```

Prompt caching is applied to the system prompt and to the last tool in the manifest, so the whole stable prefix is cached as one block.

The stub is not a convenience

A deterministic offline provider implementing the same ABC. Every agent test, every CI run and the golden-set evaluation go through it.

The argument is not cost, it is **discipline**. A test suite that costs money per run gets run less often. A non-deterministic one produces failures nobody can reproduce, which is precisely how a re-planning bug survives to production. The stub scripts responses by model type and optionally by a substring of the last user message.

What it deliberately does **not** do is pretend to reason. With nothing registered it synthesises a minimal valid instance — and a synthesised plan has *no steps*, so a test that forgot to script one fails on an empty plan rather than passing on a plausible guess.

The planner/worker split, and a defect

Found while preparing a real Claude evaluation. Worth telling.

`planner_model` was defined in settings, printed by `ari config`, shown in the Streamlit sidebar, reported by the health check as “claude (sonnet-5 / planner opus-5)”, and exposed as `planner_model_name` on both providers — and **no call site ever used it**. Every request went to `settings.model`.

That is worse than an unused setting: four surfaces told a reader the system split planning onto a stronger model, and it did not. Anyone reasoning about cost from `ari config` was reasoning about a system that did not exist.

Call	Model	Why
Plan / re-plan	planner	A bad plan wastes every tool call after it
Critic	planner	Decides whether the investigation continues
Intent classification	worker	Constrained; runs on every question
Recommendation draft	worker	Constrained by the evidence it is given

Measured: one golden-set run is **80 LLM calls — 40 planner, 40 worker** — before any re-planning.

Interview Questions

[INTERMEDIATE] Why an LLM abstraction if you only use one vendor?

Two concrete reasons, both exercised. Swapping planner and worker models independently, and running the entire test suite against a stub with no network. If neither were true I would agree the abstraction was speculative.

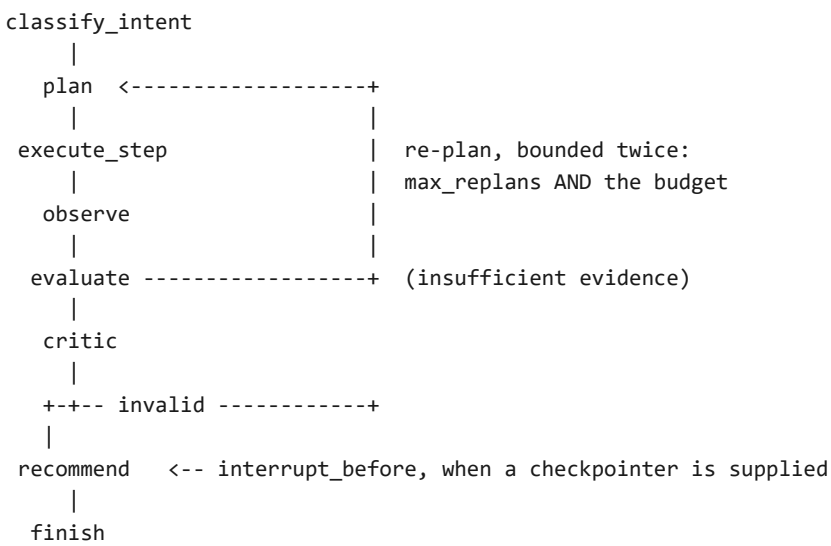
[SENIOR] How do you test an agent whose model is non-deterministic?

Separate the two questions. **Does the graph do the right thing given a response** is deterministic and tested with a scripted stub — routing, budget, re-plan bounds, guardrails. **Does the model produce good responses** is a statistical question answered by the golden set with a recorded baseline. Conflating them gives you a flaky suite that tests neither.

09 — The Agent Layer: LangGraph

Mental Model

The workflow has cycles. Plan → act → observe → critique → re-plan is a loop with a conditional exit, and the exit condition is a judgement. Chains are DAGs; that is why this is a graph.



Three agents, not four

Supervisor — intent, entity extraction, planning, tool selection, re-planning. **Critic** — validation, contradiction detection, sufficiency. **Recommendation** — synthesis and final business output.

The brief specified a fourth, Root Cause, for hypothesis generation. It was folded into the Supervisor's observe step: an agent whose only job is to interpret results already in state is a **node boundary without work behind it**, and every extra node is a round trip that has to earn itself.

The analytical models are tools, not agents, because they are deterministic. Wrapping each in an LLM would add a non-deterministic layer in front of a deterministic computation and buy latency and tokens in exchange for nothing.

State design

`AgentState` is a `TypedDict` because that is what LangGraph expects. The reducer choice is the interesting part:

- `tool_results`, `observations`, `errors`, `completed_steps` are **append-only** via `operator.add`. Evidence accumulates.
- `plan` is deliberately **replaced**, not appended. Re-planning supersedes; keeping the old plan would let a stale step execute twice.
- Results are kept as **full `ToolResult` envelopes**, not extracted numbers, because the Critic cannot judge sufficiency without the assumptions and warnings.

Separation of agents and workflows

`app/agents/` holds node logic — prompts, parsing, the decision returned. `app/workflows/` holds graph assembly — edges, routing, loops, checkpoints. They change for different reasons: tuning how the Critic judges evidence should not touch the graph, and adding a re-planning edge should not touch the Critic's prompt.

Interview Questions

[BASIC] Why LangGraph rather than a chain or plain function calling?

Cycles, explicit state, conditional edges and checkpointing. The last one is what makes human-in-the-loop possible at all — you need to interrupt, persist and resume, and a chain cannot do that.

[INTERMEDIATE] How do you stop an agent looping forever?

Two independent bounds. A `BudgetTracker` carried in state with hard caps on iterations, tool calls, tokens and wall clock. And `max_replans` on the critic edge. Two, because they fail differently: the budget catches an expensive investigation, the replan cap catches a cheap infinite one.

On breach the system returns the best recommendation the gathered evidence supports, **explicitly flagged as incomplete and capped at 0.5 confidence**. A truthful partial answer beats both an infinite loop and a confident answer built on an investigation that never finished.

[SENIOR] What is the hardest part of getting tool selection right?

It is mostly a prompt problem, not a code problem — which means you cannot tell whether a change helped without measurement. That is the entire reason the golden set exists, and the reason its most uncomfortable finding is that a keyword planner matches the model on well-posed questions.

10 — The Critic, Re-planning and Human-in-the-Loop

Mental Model

The agent whose job is to say no. It is separate from the Supervisor because an agent asked to plan an investigation and judge whether its own investigation succeeded will usually find that it did.

Splitting the roles is not ceremony. It is the only structural defence against a confident conclusion drawn from thin evidence.

Mechanical checks run first, at no token cost

Before the model is consulted, deterministic checks run over the results. A result carrying `validation_status: failed` is marked **BLOCKING** and overrides whatever the model concludes — that finding is decidable, and no confident reading of it should be able to win.

Finding	Severity	Effect
validation_status: failed	BLOCKING	Overrides the model's verdict
validation_status: warnings	raised	Caveats must travel with the number
tool returned an error	raised	Named in the verdict
partial result	raised	Warning count reported

Re-planning is bounded twice

A Critic that is never satisfied is the realistic way an agent loops forever. The cap ends the investigation, not the Critic.

And when the cap ends it, the unresolved objection **travels into the recommendation's risks and caps its confidence at 0.5**. An objection that was never answered must not arrive looking settled — that is the specific way a bounded loop would otherwise launder a failure into a clean-looking result.

Human-in-the-loop

`interrupt_before` on the recommendation node, with a checkpointer so the graph can persist and resume. **Before, not after**: the point is to review the evidence, not to rubber-stamp a conclusion already drafted.

It fires when projected impact crosses `AGENT__HUMAN_APPROVAL_THRESHOLD`, applied to the **magnitude** — recommending you give up a million is exactly as consequential as recommending you chase it.

Without a checkpointer the flag is still set but nothing blocks on it. That is the difference between "flagged for approval" and "gated on it", and I would state it rather than let it be assumed.

Interview Questions

[INTERMEDIATE] Why not have the Supervisor self-critique?

Same reason you do not review your own pull request. It has already committed to a plan and has every incentive to believe it worked. The separation is structural scepticism.

[SENIOR] The Critic rejects everything. How do you diagnose it?

First confirm it is not correct — a Critic rejecting everything on a dataset with no trained artefacts is right. Then check whether it is the mechanical layer or the model: the mechanical findings are deterministic and inspectable, so if BLOCKING is firing, the tools are genuinely returning failed validation. If it is the model, the golden set is how you tell whether a prompt change helped, and the abstention questions are where over-rejection would show up as a *score improvement*, which is the trap.

11 — Guardrails: Budget and Output Validation

The hallucination control that is architectural

The system prompt tells the model never to state a number that did not come from a tool. Output validation verifies it did not. A prompt instruction without a check is a hope.

Every numeral in the final recommendation is extracted and matched against the tool results in state, **at the rounding a readable sentence applies** — 1,427,355 written as "1.43M" passes, 0.6761 as "68%" passes.

Why it reports instead of blocking

Three reasons, all of which had to be true to justify the choice:

- **False positives are certain.** A recommendation legitimately contains a year, a horizon in days, "the top 3". Blocking on those makes the check unusable, and an unusable check gets switched off.
- **The arithmetic is real.** "150 incremental units at a 40 margin is 6,000 of profit" involves a number no tool returned. Distinguishing recomputation from invention needs reasoning the check does not have.
- **A labelled number is more useful than a missing one.** Flagging "this figure appears in no tool result" tells a reviewer where to look. Suppressing the sentence tells them nothing.

An unsourced figure caps confidence at 0.6 and is named in the output.

Two bugs found by running it

The control had a hole in exactly the case it exists for.

It missed "1.43M" entirely. The bare numeral is 1.43, which falls under the structural floor that skips years and small counts — so the commonest way of writing a large figure was never checked at all. Same hole for "68%".

Exempting suffixes and percentages then flagged "95% confidence interval" as an invented figure. Confidence levels are now scoped out by value *and* surrounding words together, so a bare "95%" is still checked.

The budget

Limit	Catches
max_iterations	A graph cycling without progress
max_tool_calls	Fan-out across every registered tool
max_token_budget	An expensive investigation
max_execution_seconds	A hung tool

Four independent limits because they fail differently. A token cap does not catch a fast infinite loop; an iteration cap does not catch one enormous call.

What is deliberately not built

- **SQL validation** — agents never author SQL. Tools take typed Pydantic inputs and the repository owns every query, so there is no injection surface.
- **Prompt-injection screening** — screens retrieved documents, and there is no document corpus.
- **Role-based permissions** — `ToolSpec` carries a permission and the registry can filter on it, but nothing populates a caller role. The filter is the mechanism; authentication is the gap.
- **PII filtering** — the dataset is synthetic and carries no personal data.

Interview Questions

[BASIC] How do you stop the LLM inventing numbers?

Four layers, and only one is a prompt. The prompt forbids it. Structured output via forced tool-calling constrains the shape. **Post-hoc validation checks every numeral against the tool results.** And the tools refuse rather than guess — the uplift service returns a structured refusal with a `recoverable` flag instead of a number when the causal assumptions fail.

[SENIOR] Your validator has false positives. Does that not make it useless?

It would if it blocked. It reports, and the report names the figure and the sentence it appeared in, so a reviewer resolves it in seconds. The alternative — a blocking check tuned to avoid false positives — would have to be so permissive that it missed real inventions. I would rather over-flag into a human's field of view than under-flag into a board pack.

12 — Agent Evaluation: The Golden Set

The chapter that makes every other chapter defensible.

Mental Model

Every earlier step scored a model against ground truth. This scores the agent: whether it picks the right tools, gathers what it needs, reads the evidence the way the truth points, and declines when it cannot answer.

The questions are derived, not invented

Twenty questions built from the scenarios the generator injected. That record is the only place a right answer exists **independently of the thing being graded** — a question written separately would be scored against an expectation written separately, which measures nothing.

Nine of the twenty are unanswerable on purpose

This is the design decision most benchmarks omit.

No registered tool diagnoses a stockout, a competitor price cut or a lost-distribution shock. For those the correct behaviour is to **decline**, and confidence is the failure. They are scored on abstention instead of accuracy.

Dropping them would have hidden a real coverage gap behind a better-looking score. A system that answers everything confidently is worse than one that knows its own coverage, because the second can be trusted about the first.

Label	Count	Scored on
successful_promo	3	tool selection, evidence, direction
bad_promo	3	… plus the trap (see below)
price_increase	3	tool selection, evidence, direction
seasonal_peak / product_launch	2	minimum-sufficient workflow
stockout	4	abstention
competitor_price_cut	4	abstention
regional_shock	1	abstention

Four dimensions, never averaged into one

They fail for different reasons and a single score hides which. An agent that selects perfect tools and then writes an unsupported conclusion should not score the same as one that picks badly and reports honestly.

- **Tool selection** — required tools called, with a penalty for fan-out. Calling everything looks thorough and is the cheapest way to appear rigorous without being it.
- **Evidence** — did the calls actually produce usable results?
- **Direction** — does the finding point the way the truth points? Sign only, never magnitude; magnitude is the estimators' business.

- **Abstention** — on an unanswerable question, did it decline?

The floor is a keyword planner

A stub run grades whatever the stub was scripted to return — script the right answer for every question and you score 100%, which measures the person who wrote the script. So the stub follows a **policy** instead: regex over the question text, no reasoning of any kind.

Dimension	Keyword floor
answerable mean	0.833
abstention mean	0.000
tool selection	1.000
evidence	0.727
direction	0.773

Say the uncomfortable row out loud

Keyword routing MATCHES a language model on tool selection for well-posed single-capability questions. The LLM does not earn its place there.

Where it does earn it is the other two rows. The floor scores **0.000 on abstention**, because knowing your own coverage requires judgement and regex has none. And it takes half marks on the `bad_promo` trap, where uplift is genuinely positive and the right answer is still *do not repeat it* — the floor says “proceed on the evidence gathered” every time.

Volunteering a result that undercuts your own system is the single most senior thing available in this document, and it is true.

Two scorer bugs, both measuring the wrong thing

Scoring direction on incremental profit graded Step 7's spend artefact, not the agent. Spend runs ~20× achievable margin at this grain, so profit is negative even for promotions injected as successful — all three scored zero. The scenario record fixes the *volume* sign, so volume is what gets read. That change alone moved `answerable_mean` from 0.727 to 0.833.

Scoring the trap from the tool's ROI field graded Step 7 again. The decision not to repeat exists only in the recommendation, so that is where it is read from. Before the fix the keyword baseline scored a perfect 1.0 on the one question designed to be failable.

Artefact gaps are separated from planning errors

A required tool that ran and found no trained series for that product is a **coverage** problem with a coverage fix — retrain over the products the questions ask about. Reading it as poor reasoning sends the effort somewhere it cannot help. Currently 3 of 11 answerable questions.

Interview Questions

[INTERMEDIATE] How do you evaluate a non-deterministic agent at all?

You do not evaluate the text. You evaluate decisions with known correct answers — which tools, which direction, whether to answer at all — and you record a baseline so a prompt change is measurable rather than a matter of opinion. Baselines are per-provider and the comparison **refuses** across providers, because grading a Claude run against the keyword floor would report noise as improvement.

[SENIOR] What is wrong with your own evaluation?

Have an answer. There is one.

The trap check is **keyword matching over the recommended action**, so a conclusion that argues against repeating in words my list does not contain scores as though it argued for it. I took that over a model grading a model, because a scorer whose errors correlate with the system's is worse than a blunt one. It is a real limitation and I would fix it with a labelled set before trusting the dimension across providers.

Second: the stub run does not measure the model at all. It measures the harness and the keyword policy. Only a Claude run measures capability, and that number is **not yet recorded** — so I would not quote one.

13 — API, State and UI

One service behind three interfaces

The CLI, the API and the Streamlit UI all call one `InvestigationService`, so a question answered in the terminal is answered identically over HTTP. **The UI talks HTTP rather than importing the container** — the shortcut would make it a second consumer of the internals rather than a client of the API, and an endpoint could break without the demo noticing.

Three decisions worth defending

An investigation that gathered no usable evidence is `failed`, not `completed` — even though the graph ran to the end without raising. `completed` is a claim that the question was answered, and reporting an empty result that way is how "we found nothing" becomes "there is no effect".

A failed investigation returns 200 with a `failed` status, not a 5xx. The request was handled correctly; the investigation is what did not conclude. That distinction is what lets a caller tell "the platform is down" from "the evidence did not support an answer".

`POST /scenario` reports the levers it could not model. The API accepts an inventory change and a promotion-spend amount the engine cannot project; both come back as warnings rather than being dropped, because a projection that silently ignored the change the caller asked about would answer a different question than the one posed.

The trace

Reconstructed from the finished state, **not emitted during the run**. Instrumenting every node with a callback would couple the graph to a sink, and the sink would then be the thing that must not fail — a trace writer throwing mid-investigation would lose the investigation. What it gives up is per-node wall-clock timing, which nothing currently asks for.

The bug: the service minted a trace_id, stored it, then let the graph mint a different one — so the stored trace and the returned outcome could not be joined. A silent failure, because both ids look valid.

Storage: two engines, one job each

	DuckDB + Parquet	SQLite
Holds	Business data	Investigations, traces, feedback
Shape	Columnar, scan-heavy	Row, write-heavy
Rows	23.6M	One per question asked

The recommendation is stored as JSON rather than normalised across five tables. It is always read whole, and normalising would buy join flexibility nobody needs at the cost of a migration every time the model gains a field. The trade-off is that it is not SQL-queryable — accepted, because the question asked of this store is "what did investigation X conclude", never "find every recommendation mentioning margin".

Feedback is deliberately NOT foreign-keyed to investigations and survives a purge. It is the only human-labelled signal this platform produces, and a cascade delete would destroy the scarcest data here.

14 — The Defence: Limitations and Trade-offs

Mental Model

Volunteer your limitations before you are asked. Senior engineers are trusted because they mark their own work honestly, and almost nobody does it in interviews.

What is genuinely absent

Absent	Why, honestly
Databricks / Stage 2	Designed with raising bodies. Not built.
Agentic RAG	A decision, not a gap. There is no document corpus — every number comes from a model over structured data, so a vector store would be scaffolding for its own sake.
Azure, Neo4j, MCP, LangSmith	Not present. Do not claim them.
Async job queue	Investigations run synchronously.
Authentication	The permission filter exists; nothing populates a role.
A verified Docker build	Dockerfile and compose are written; the lockfile resolves against the base image and the compose file parses, but Docker is not installed on the dev machine so the image has never been built.
A recorded Claude evaluation score	Only the keyword floor is committed.

The five things to remember

- **LLM reasons, deterministic models compute.** Never blur it.
- **+34.9% of pure fiction** — the naive method on promotions that did nothing at all.
- **0.7 percentage points** — ground-truth recovery across 4,417 events.
- **The −424% story** — shows you debug rather than accept.
- **"A keyword planner ties the LLM on tool selection"** — volunteer the result that undercuts your own system.

What I would do next, in order

- Record a Claude golden-set baseline, so the capability claim has a number behind it rather than a floor and an inference.
- Retrain forecast and uplift artefacts over the products the golden set asks about, closing the 3-of-11 artefact gap.
- Replace the trap's keyword check with a small labelled set, so that dimension survives a provider change.
- Build the image and run the compose stack, then delete the caveat from the README rather than leaving it as a permanent asterisk.
- Authentication behind the existing permission filter — the mechanism is there and unused, which is the cheapest real security win available.

The tone that wins

> Say “the ROI numbers on this dataset are not interpretable and here is why” before they find it. Say “ignorability holds here because the generator's targeting is observable — that is a property of the data, not an achievement.” Say “the Docker image is written but I have never built it.”

The trap in the other direction: now that the system is built, do not underclaim. Describe it in the present tense, then be precise about the edges.